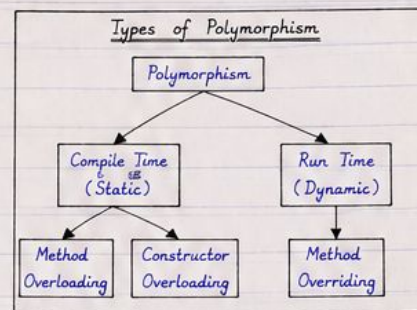


1. Polymorphism (पॉलीमॉर्फिज्म):

- Polymorphism का मतलब होता है "Many Forms" (कई रूप).
- Java में polymorphism के द्वारा एक ही method या constructor अलग-अलग conditions में अलग-अलग तरीके से काम करता है।
- Polymorphism को achieve करने के कई तरीके हैं, जिनमें से एक है Overloading.
- Overloading में same class के अंदर same नाम वाले methods या constructors को different parameters के साथ define किया जाता है।



* (A) Method Overloading (मेथड ओवरलोडिंग):

- जब एक ही class के अंदर same नाम के एक से अधिक methods को different parameter list के साथ define किया जाता है, उसे Method Overloading कहते हैं।
- Return type अलग होने पर केवल overloading नहीं होता, parameters अलग होने चाहिए।
- Method overloading compile time (static) polymorphism को achieve करता है।

Syntax:

```

returnType methodName(parameter list 1) {
    // code
}

returnType methodName(parameter list 2) {
    // code
}

returnType methodName(parameter list n) {
    // code
}
  
```

Example:

```

class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    double add(double a, double b) {
        return a + b;
    }
}
  
```

Test Class:

```

class Test {
    public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println(c.add(10, 20)); // 30
        System.out.println(c.add(10, 20, 30)); // 60
        System.out.println(c.add(10.5, 20.5)); // 31.0
    }
}
  
```

Note: Method overloading में method का नाम same होता है, लेकिन parameter list (number, type या order) different होता है।

* (B) Constructor Overloading (कन्स्ट्रक्टर ओवरलोडिंग):

- जब एक ही class के अंदर same नाम के एक से अधिक constructors को different parameter list के साथ define किया जाता है, उसे Constructor Overloading कहते हैं।
- Constructor overloading का use object को अलग-अलग तरीके से initialize करने के लिए किया जाता है।
- Constructor overloading भी compile time (static) polymorphism को achieve करता है।

Syntax:

```

ClassName() {
    // code
}

ClassName(parameter list 1) {
    // code
}

ClassName(parameter list 2) {
    // code
}
  
```

Example:

```

class Student {
    int id;
    String name;
    double marks;

    // Constructor 1 (no-arg)
    Student() {
        id = 0; name = "Unknown"; marks = 0.0;
    }

    // Constructor 2 (2 parameters)
    Student(int id, String name) {
        this.id = id;
        this.name = name;
        this.marks = 0.0;
    }

    // Constructor 3 (3 parameters)
    Student(int id, String name, double marks) {
        this.id = id;
        this.name = name;
        this.marks = marks;
    }
}
  
```

Test Class:

```

class Test {
    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = new Student(101, "Aman");
        Student s3 = new Student(102, "Neha", 87.5);

        System.out.println(s1.id + " " + s1.name +
            " " + s1.marks); // 0 Unknown 0.0
        System.out.println(s2.id + " " + s2.name +
            " " + s2.marks); // 101 Aman 0.0
        System.out.println(s3.id + " " + s3.name +
            " " + s3.marks); // 102 Neha 87.5
    }
}
  
```

Note: Constructors का नाम class के नाम के same होता है, return type नहीं होता (यहाँ तक कि void भी नहीं)।

* Method Overloading vs Constructor Overloading

Sr. No.	Point	Method Overloading	Constructor Overloading
1.	Belongs to	Methods of the class	Constructors of the class
2.	Purpose	Same name के methods को different parameters के साथ define करना	Object को अलग-अलग तरीके से initialize करना
3.	Return Type	Return type हो सकता है (void या non-void)	Return type नहीं होता (यहाँ तक कि void भी नहीं)
4.	Name	Method name same होता है।	Constructor name class name के same होता है।
5.	Parameters	Number, type या order अलग होना चाहिए	Number या type अलग होना चाहिए
6.	Polymorphism Type	Compile time (Static) Polymorphism	Compile time (Static) Polymorphism

1. Polymorphism (पॉलीमॉर्फिज्म):

→ Polymorphism का मतलब होता है "Many Forms (कई रूप)"।

→ Java में polymorphism के दो main प्रकार होते हैं:

1. Method Overriding (Runtime polymorphism)
2. Up-casting and Down-casting

(A) Method Overriding (मैथड ओवरराइडिंग):

- जब child class, parent class के method को same signature (नाम, parameter list) के साथ re-define करती है, उसे Method Overriding कहते हैं।
- यह Runtime polymorphism का उदाहरण है।
- इसमें method call, object के actual type के आधार पर resolve होता है।

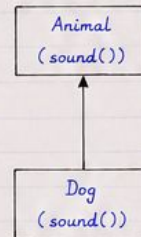
Syntax:

```
class Parent {  
    void display() {  
        // code  
    }  
}  
  
class Child extends Parent {  
    @Override  
    void display() {  
        // code  
    }  
}
```

Example:

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
class Test {  
    public static void main(String[] args) {  
        Animal a = new Dog();  
        a.sound(); // Dog barks  
    }  
}
```

Diagram:



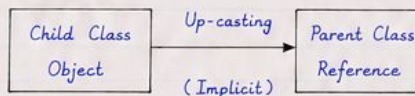
Dog class ने sound() method को override किया।

Note: Method Overriding में "IS-A" relationship और inheritance जरूरी है।

(B) Up-casting (अप-कास्टिंग):

- जब child class के object को parent class reference में assign किया जाता है, उसे Up-casting कहते हैं।
- यह implicit (automatic) होता है, इसलिए इसे implicit up-casting भी कहते हैं।
- Up-casting में child class के सभी overridden methods accessible रहते हैं।

Diagram:



Example:

```
class Parent {  
    void show() { System.out.println("Parent"); }  
}  
  
class Child extends Parent {  
    @Override  
    void show() { System.out.println("Child"); }  
}  
  
class Test {  
    public static void main(String[] args) {  
        Parent p = new Child(); // up-casting  
        p.show(); // Child  
    }  
}
```

Note:

Up-casting में child class के specific members (जो parent में नहीं हैं) access नहीं होते।

(C) Down-casting (डाउन-कास्टिंग):

- जब parent class reference को वापस child class reference में convert किया जाता है, उसे Down-casting कहते हैं।
- यह explicit (manual) होता है, इसलिए इसे explicit down-casting कहते हैं।
- Down-casting करने से पहले object वास्तव में child class का होना चाहिए, नहीं तो ClassCastException आएगा।

Diagram:



Example:

```
class Parent {  
    void show() { System.out.println("Parent"); }  
}  
  
class Child extends Parent {  
    void display() { System.out.println("Child Display"); }  
}  
  
class Test {  
    public static void main(String[] args) {  
        Parent p = new Child(); // up-casting  
        Child c = (Child) p; // down-casting  
        c.show(); // Parent  
        c.display(); // Child Display  
    }  
}
```

Important Note:

- ✓ Down-casting हमेशा manual होता है (type cast के साथ)।
- ✓ पहले object का type check करना safe होता है।
- ✓ instanceof operator का use करके check करते हैं।

Example Check:

```
if (p instanceof Child) {  
    Child c = (Child) p;  
    c.display();  
}
```

Quick Summary:

- Method Overriding → Same method, same signature, child class द्वारा re-define, Runtime polymorphism।
- Up-casting → Child object को parent reference में, Implicit, safe।
- Down-casting → Parent reference को child reference में, Explicit, पहले check करना जरूरी।